STUDY & EXAM PREPARATION GUIDE

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 18

Collections API

CHAPTER OVERVIEW

This chapter covers the following exam objectives: Create List, Set, Map, and Deque collections, and add, remove, update, retrieve and sort their elements, Sort array elements

EXAM OBJECTIVES COVERED

- Create List, Set, Map, and Deque collections, and add, remove, update, retrieve and sort their elements
- Sort array elements

Chapter 18 Collections API

Exam Objectives

- Create List, Set, Map, and Deque collections, and add, remove, update, retrieve and sort their elements
- Sort array elements

18.1 Overview of the Collections API ☝

The Java collections framework contains more than a hundred classes and interfaces organized within the **java.util** package. If that seems a lot, let me tell you that there are third party collection libraries also that exist for the same purpose, such as Apache Commons Collections and Google Guava and those are quite popular too! The reason for so many utility libraries available in the Java world is that there really are many variations of standard data structures and algorithms. New data structures and algorithms are also routinely developed in response to the challenges faced by software developers. Classes provided by one library may not be enough for a project. For example, in one of the projects, I wanted to use a map where the entries would be removed automatically if they are unused for a certain duration. I used PassiveExpiringMap from Apache Commons collection because the Java collections framework does not have anything like that.

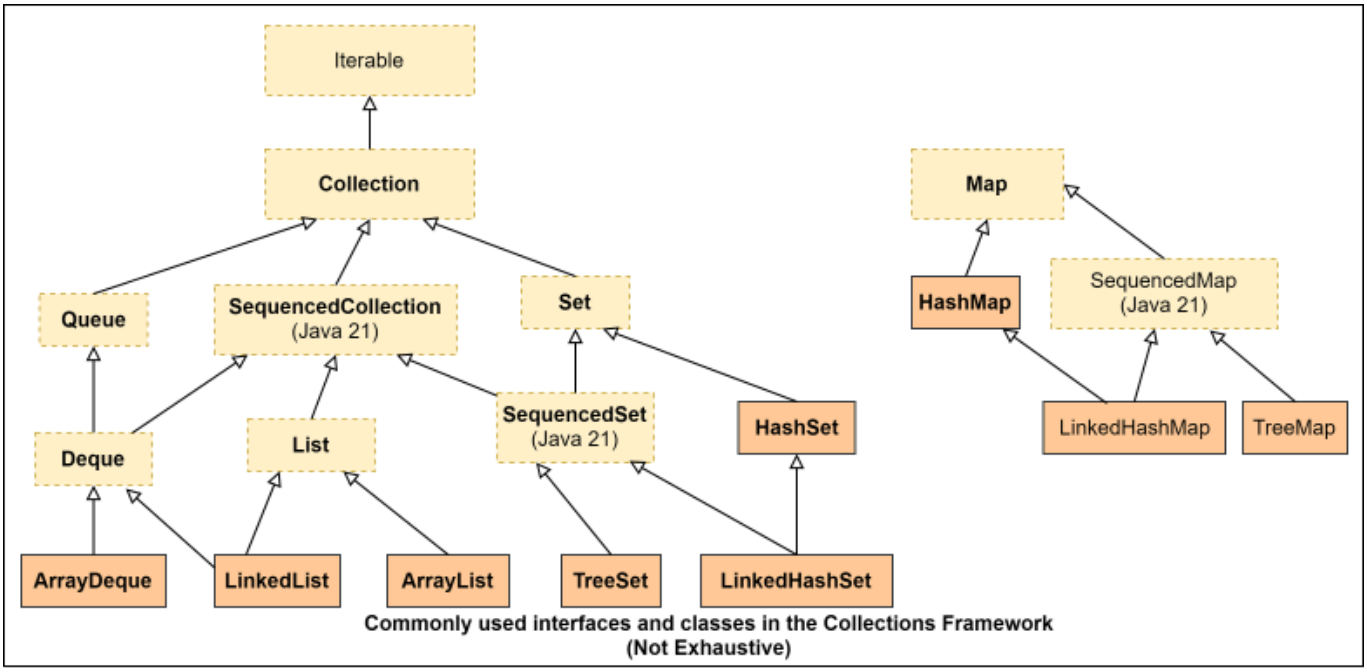
To its credit, the Java collections framework provides a well thought out collection of commonly required base interfaces and classes, and since it is included in the JRE, one will be hard pressed to find a project that doesn't use this framework. Even classes from other frameworks rely on the base interfaces and classes of the Java collections framework. For example, the PassiveExpiringMap class I mentioned above implements the `Map` interface from the Java collections framework. It is, therefore, important for a Java developer to have a good understanding of this library.

Since it is practically impossible to remember all available classes and their methods, the best way to master this API is to understand the general theme of the library, know its root interfaces and a few commonly used implementation classes, and the implementation quirks that are common in their methods. This will give you enough of an idea about where to look to find a class that meets your requirement whenever the need arises. If you are not able to find anything that meets your requirements in the Java collections framework, you may look into third party libraries.

The OCP exam also follows the same strategy and focuses mostly on the basics. Since the official exam objectives do not specify an exhaustive list of classes and methods, I will go into the details of only those classes and methods that I know from experience (and from the feedback that we get from Enthuware's online/offline trainees who have taken the exam), are on the exam. Most of the classes and methods behave the way one might expect from their names and so, even if you get a question on a method that you have not seen before, you will be able to figure out the answer using your basic API knowledge and common sense.

Interfaces and Classes in the Collections Framework

The following diagram shows some of the important and most used types available in the collections framework. The rectangles with dashed borders are interfaces and the rectangles with solid borders are classes. The ones mentioned in bold are the ones you will most likely see on the exam. The rest are also officially on the exam and you may see them used in code snippets but we haven't seen anyone getting questions that require specific knowledge of those.



Observe the following points in the above diagram.

1. There are two root interfaces - `Iterable` and `Map`. They form two independent trees of collection types in the Collections Framework.
2. The word **Collection** (with an upper case C) generally refers to a class that implements the `Collection` interface but the word **collection** (with a lower case c) colloquially includes `Collection`s as well as `Map`s because `Map` has methods that allow us to access its keys and values as `Collection`s.
3. The framework includes interfaces as well as classes that implement those interfaces.
4. Prior to Java 21, there was no interface to represent an ordered collection and so, it was not possible convey the requirement of "order" through an interface. You had to use either a `List` or a `SortedSet`. `SequencedCollection` was added in Java 21 to fill this lacuna. Note that it imposes no restriction on how the elements are actually stored in memory. All it mandates is that if you iterate through the elements then you will get the elements in a well defined order.

The following tables summarize the interfaces and classes that you are required to know for the exam. You are not required to remember all of their methods by heart but you should know at least the ones I have included below. Again, I strongly encourage you to go through their JavaDoc descriptions.

Collections Framework Interfaces Summary

Interface	Features	Notable methods	Usage Notes
`Collection` extends `Iterable`	1. Silent on order, uniqueness, and nullability of its elements 2. No direct implementing class	**Single element operations**: `add`, `remove` **Bulk operations**: `addAll`, `removeAll`, `removeIf`, `retainAll`, `clear` **Inspection operations**: `size`, `contains`, `containsAll`, `isEmpty`	Commonly used as a parameter type or a return type of a method.
`SequencedCollection` extends `Collection`	1. Well defined encounter order 2. Silent on uniqueness and nullability of its elements	**Operations to access/mutate first and last elements**: `addFirst`, `addLast`, `removeFirst`, `removeLast`, `getFirst`, `getLast` **Provides reverse view**: `reversed`	Added in Java 21 to represent ordered collections

`List` extends `SequencedCollection`	1. Ordered 2. Duplicate and `null` values are usually allowed but depends on implementation	`add`, `remove`, and `set` methods with `int` index as the first argument, `subList` Static factory methods `of` and `copyOf` to create unmodifiable `List`s	Used when precise control over where in the list each element is inserted is required.
`Queue` extends `Collection`	1. Ordered 2. Duplicates usually allowed 3. Nulls are usually not allowed	`offer`, `poll`, `peek`	Used when rule based ordering such as FIFO, LIFO, or priority is needed.
`Deque` extends `Queue` and `SequencedCollection`	1. Ordered 2. Duplicates usually allowed 3. Nulls are usually not allowed	`offerFirst`, `offerLast`, `pollFirst`, `pollLast`, `peekFirst`, `peekLast`	Used when element insertion and removal at both ends is required.

`Set` extends `Collection`	1. Unordered 2. No duplicates 3. One `null` value may be allowed depending on the implementation	Static Factory methods named `of` and `copyOf` to create unmodifiable `Set` s	Used when duplicates are to be avoided and order is not important.
`Map` **does not extend** `Collection`	1. Stores key-value pairs 2. No duplicate keys 3. Silent on `null` key and `null` values 4. Silent on order of elements	`get` , `getOrDefault` , `put` , `putAll` , `putIfAbsent` , `remove` , `keySet` , `entrySet` , `values` , `replace` , `containsKey` , `forEach` Static Factory methods named `of` , `ofEntries` , and `copyOf` to create unmodifiable `Map` s	Used when key value pairs are to be stored
`SequencedMap` **extends** `Map`	1. Well defined encounter order of entries 2. supports operations at both ends 3. reversible	`firstEntry` , `lastEntry` , `pollFirstEntry` , `pollLastEntry` , `putFirst` , `putLast` , `reversed` , `sequencedEntrySet` , `sequencedKeySet` , `sequencedValues`	Used when key value pairs need to be retrieved in a defined order

Collections Framework Implementation Classes Summary

Class	Features	Usage Notes

`ArrayList` implements `List`	1. Ordered based on index 2. Allows duplicates and `null` values 3. Modifiable	1. Backed by an array 2. Fast access but slow insertion 3. **Constructors:** `ArrayList()`, `ArrayList(int initialCapacity)`, `ArrayList(Collection c)`
`LinkedList` implements `List` and `Deque`	1. Ordered based on index 2. Allows duplicates and nulls 3. Modifiable	1. Backed by a doubly-linked list 2. Fast insertion but slow access 3. Nulls are allowed but advised not to add nulls because `Queue`'s `poll()` method returns `null` to indicate empty queue 4. **Constructors:** `LinkedList()`, `LinkedList(Collection c)`

Lists returned by `List`'s `of` and `copyOf` methods	1. Unmodifiable 2. No nulls. NPE if input contains `null`. 3. Independent from the input array	Example: `List<String> ls = List.of("a", "b");`
List returned by `Arrays.asList` method	1. Unmodifiable 2. `null` and duplicate values are allowed 3. Backed by the underlying array	Example: `List<Integer> li = Arrays.asList(new Integer[]{1, null});`
`ArrayDeque` implements `Deque`	1. Ordered based on insertion order 2. Allows duplicates 3. No nulls	1. Backed by an array 2. Adding `null` throws NPE. 3. Supports FIFO as well as LIFO 4. **Constructors**: `ArrayDeque()`, `ArrayDeque(int numElements)`, `ArrayDeque(Collection c)`

`HashSet` implements `Set`	1. Unordered 2. Duplicates ignored 3. Allows `null`	1. Based on a hash table. 2. Uses `equals` and `hashCode` method to check for duplicates while adding 3. **Constructors**: `HashSet()`, `HashSet(int initialCapacity)`, `HashSet(Collection c)`, `HashSet(int initialCapacity, float loadFactor)`
`TreeSet` implements `SequencedSet`	1. Ordered 2. Duplicates ignored 3. No nulls 4. Although it implements `SequencedCollection`, it throws `UnsupportedException` from methods such as `addFirst` / `addLast`.	1. Based on a `TreeMap`. 2. Use `Comparable` or `Comparator` to check for duplicates while adding 3. **Constructors**: `TreeSet()`, `TreeSet(Comparator c)`, `TreeSet(Collection c)`, `TreeSet(SortedSet s)`

Sets returned by `Set`'s `of` and `copyOf` methods	1. Unmodifiable 2. `of` throws IAE if input contains duplicates but `copyOf` silently ignores duplicates. 3. No `null`. NPE if input contains `null`.	Example: `Set<String> ls = Set.of("a", "b");`
`HashMap` implements `Map`	1. Unordered 2. Allows `null` as a key and also as a value	1. Based on a hash map 2. **Constructors**: `HashMap()`, `HashMap(int initialCapacity)`, `HashMap(int initialCapacity, float loadFactor)`, `HashMap(Map m)`
`TreeMap` implements `SortedMap` (Not too important for the exam.)	1. Ordered 2. Sorted 3. `null` value is allowed but not key	1. Based on Red-Black tree 2. **Constructors**: `TreeMap()`, `TreeMap(Comparator c)`, `TreeMap(Map m)`, `TreeMap(SortedMap m)` 3. Has many additional methods due to the fact that it implements `SortedMap`

Maps returned by `Map`'s `of`, `ofEntries`, and `copyOf` methods	1. Unmodifiable 2. Unordered 3. `null` key or value not allowed	Example: `Map<String, String> m = Map.of("key1", "value1");`

Exceptions thrown by collection methods

As you can see from the tables above, implementation classes may impose their own constraints besides the ones that are specified through their interface definitions. For example, `ArrayDeque` does not allow `null` and unmodifiable lists do not allow addition/removal, and so on. Since such constraints cannot be expressed in the interfaces or method signatures, the compiler cannot check for their violations in the code at compile time. Such violations are checked at run time through the use of the following five **unchecked exceptions**:

1. `UnsupportedOperationException`: If the operation is not supported by this collection. For example, trying to add an element in a list created using the `List.of` method will cause an `UnsupportedOperationException` to be thrown at run time.
2. `NullPointerException`: If the specified element is `null` and this collection does not permit `null` elements.
3. `IllegalArgumentException`: If some property of the element prevents it from being added to this collection.
4. `ClassCastException`: If the class of the specified element prevents it from being added to this collection
5. `IllegalStateException`: If the element cannot be added or removed at this time due to insertion/deletion restrictions.

Since this information is available only in the JavaDoc description of the class/method, it is important to go through it before using a that class/method.

The exam does not require you to know all the scenarios that will cause the above exceptions to be thrown but it expects you to detect if an `UnsupportedOperationException` or a `NullPointerException` will be

thrown in the given code.

18.2 Accessing collections ☝

Since every `Collection` is an `Iterable`, the most uniform way to process the elements of a `Collection` is to use the methods of the `Iterable` interface.

Iterable

You saw the "enhanced for loop" earlier in the **Using Loop Constructs** chapter. The `java.util.Iterable` interface was introduced in Java 1.5 to make collections compatible with the enhanced for loop without breaking existing `Collection` implementations. It has the following three methods, each of which gives us a different way to iterate through the elements of a collection:

1. `Iterator iterator()`: Returns an object of type `Iterator`. Before the introduction of the enhanced for loop, this was the most common way of iterating through the elements of a collection.
2. `void forEach(Consumer<? super T> action)`: This method provides the modern and the preferred way of processing each element of a collection. Recall that `Consumer` is a functional interface and so you will see this method getting called with **lambda expressions** a lot.
3. `default Spliterator<T> spliterator()`: It allows splitting a collection into multiple parts so that each part can be processed in parallel.

The following code shows how to iterate through any `Iterable`:

```
List<Integer> listI = List.of(1, 2, 3, 4);

//using an Iterator
Iterator<Integer> it = listI.iterator();
while(it.hasNext()) System.out.println( it.next());
```

```
//using the enhanced for loop
for(Integer i : listI) System.out.println(i);

//using forEach with lambda
listI.forEach( (i)→ System.out.println(i) );

//using forEach with a method reference
listI.forEach(System.out::println);
```

Iterator

The usage of `hasNext()` and `next()` methods shown in the above code is idiomatic of Iterators. You call the `hasNext()` method to see if there is still an element remaining to be processed and if so, you call `next()` to get that element. Calling `next()` if there are no elements left, causes a `NoSuchElementException` to be thrown.

Besides `hasNext` and `next`, `Iterator` has the follow two methods:

1. `default void forEachRemaining(Consumer<? super E> action)`: Performs the given action for each remaining element until all elements have been processed or the action throws an exception.
2. `default void remove()`: This method is useful if you want to remove the element you just retrieved using a call to `next()` from the underlying collection. This method can be called only once per call to `next()`.

Spliterator

A `Spliterator` is an `Iterator` on steroids. Although it is not seen used too often, I prefer it over an `Iterator` because it is quite powerful and useful in processing elements of a collection or a stream sequentially as well as concurrently, which is most likely why the exam has questions on it. It seems a bit complicated but it is not. Let's start with something simple:

```
List<String> names = List.of("a", "b", "c", "d", "e");
Spliterator<String> si = names.spliterator();
si.forEachRemaining(System.out::print); //prints abcde
```

At the time of the creation of the `Spliterator`, it contains all the elements of the collection, and so you can say that every element of the collection is "remaining" to be processed. Thus, the `forEachRemaining` method executes the `Consumer` function for every element of the collection, thereby printing `abcde`. Nothing too exciting. You can do the same with `Iterator`'s `forEachRemaining` method.

But if you want to process just one element from a collection, you may use `Spilterator`'s `tryAdvance(Consumer c)` method:

```
List<String> names = List.of("a");
Spliterator<String> si = names.spliterator();
boolean gotOne = si.tryAdvance(System.out::print); //prints a, returns true
gotOne = si.tryAdvance(System.out::print); //no exception thrown, returns false
```

There is no exception even if you call `tryAdvance` multiple times over an exhausted or an empty collection. It just returns `false` if there is no element left to advance to. This is an improvement over `Iterator` because an `Iterator` requires two calls - `hasNext` and `next` to do the same.

The real power of a `Spliterator` is in its ability to partition out half of the elements that are remaining to be processed into a separate `Spliterator`. This is done using the `trySplit` method:

```
List<String> names = List.of("a", "b", "c", "d", "e");
Spliterator<String> split1 = names.spliterator();
Spliterator<String> split2 = split1.trySplit();
```

Now, we have two `Spliterator`s, each containing about half of the elements in the original list. Both the Spliterators can process the elements independently like so:

```
split1.forEachRemaining((value) → System.out.print("S1 "+value+", "));
System.out.println();
split2.forEachRemaining((value) → System.out.println("S2 "+value+", "));
```

The output of the above code is:

```
S1: c, S1: d, S1: e,
S2: a, S2: b,
```

The above output shows that the first `Spliterator` hived off elements `a` and `b` to the second `Spliterator` and was left with elements `c`, `d`, and `e`. The exact division of elements is not always predicable. How a `Spliterator` partitions the elements depends on how the `Spliterator` is coded by the class of the underlying collection. Although every detail of the implementation cannot be expressed programmatically, important features of a `Spliterator` can be inspected programmatically using a few `static` `final` `int` constants and the `hasCharacteristics(int characteristics)` method defined in `Spliterator`. For example, `split1.hasCharacteristics(Spliterator.ORDERED);` returns `true`, which means that this `Spliterator` has a definite order in which it keeps its elements. This is expected because the underlying collection in this example is a `List`, which is ordered. If you run the same example using a `HashSet` instead of a `List`, it will print `false`.

You can also check if a `Spliterator` knows the exact number of elements in it by passing `Spliterator.SIZED` to `hasCharacteristics` and if not, you may call `estimateSize()` to get an estimate. Knowing the size is helpful in deciding whether you want to call `trySplit` on the `Spliterator` again.

The above example is only meant to illustrate how to split a collection using a `Spliterator`. There is no benefit in splitting a collection in a situation like above. Splitting is only beneficial when the partitioned elements are processed concurrently. I will show you an example while discussing concurrency later.

Accessing SequencedCollections

`SequencedCollection` provides access to first and last elements through its `getFirst()` and `getLast()` methods. These methods throw `NoSuchElementException` if the collection is empty. It also has a `reversed()` method that provides a reverse view of the collection..

Accessing Lists

`List` provides positional/indexed access to its elements using its `get(int index)` method. It throws `IndexOutOfBoundsException` if an out of range index is passed. Remember that `List` is a `SequencedCollection` and so it too has the `getFirst()` and `getLast()` methods.

Accessing Sets

`Set` declares no new methods and so its elements can be accessed only by first acquiring an `Iterator` or a `Spliterator`. However, since a `TreeSet` is sorted (and is therefore, ordered), it provides a few additional methods to access first and last elements. You may check out the JavaDoc for those methods but they are not important for the exam.

Accessing Queues and Deques

`Queue` and `Deque` have a large number of methods for their manipulation besides the ones provided by `Collection`. Although I haven't seen anyone getting a question on

them, you should be aware of one important characteristic about their methods. Their methods can be categorized into two flavors based on how they handle the situation where there is no result. Some of the methods throw an exception when there is no result, while others return `false` or `null`. For example, in the following code, `q.element()` throws `NoSuchElementException` because the queue is empty but `q.poll()` returns `null`:

```
Queue<String> q = new ArrayDeque<>();
String head = q.poll(); //returns null
head = q.element(); //throws NoSuchElementException
System.out.println(head);
```

I am not presenting a categorized list of all the methods here because the exam does not expect you to remember this detail for each method.

Accessing Maps

Once you get the `Set` of a map's keys using the `keys()` method, or its key-value pairs using the `entrySet()` method, or the `Collection` of its values using the `values()` method, you can iterate through them just like you iterator through any other `Collection`:

```
Map<String, String> m = new HashMap<>();
m.put("k1", "v1");
Set<Map.Entry<String, String>> entries = m.entrySet();
for(Map.Entry<String, String> me : entries){
   System.out.println(me.getKey()+" "+me.getValue());
}
```

Or you can use a lambda to implement a `BiConsumer` and pass it to `Map`'s `forEach(BiConsumer bc)` method:
`m.forEach((key, value)→System.out.println(key+" "+value));`

Don't be alarmed by the weird looking `Map.Entry` type in the above code. `Entry` is a static interface defined inside `Map` and it provides methods to access and mutate a single key-value pair of the map.

Of course, if you have the key, you may use `Map`'s `get(Object key)` or `getOrDefault(Object key, Object defaultValue)` methods to retrieve the associated value. The `get` method returns `null` if the key is not found and `getOrDefault` returns `defaultValue`. Since `HashMap` allows `null` values, a return value of `null` may not mean that the key is not present in the map. It may also mean that the key is associated with `null`. To distinguish between the two situations, you need to use the `containsKey` method, which returns `true` only if the key is found in the map.

Accessing SequencedMaps

Just like `SequencedCollection`, `SequencedMap` has `firstEntry()` and `lastEntry()` methods to get the first and the last entries in the map without removing them from the map. They return `Map.Entry<K,V>` objects. It also has `pollFirstEntry()` and `pollLastEntry()` methods that do the same thing but they remove the returned entries from the map. However, unlike `SequencedCollection`, these methods just return `null` if the map is empty.
It has a `reversed()` method that provides a reverse ordered view of the map.

18.3 Mutating collections ☝

Adding and removing elements to and from a `Collection` can be done using `add`, `allAll`, `remove`, `removeAll`, `removeIf`, `retainAll`, and `clear` methods declared in `Collection`. Specialized collections such as `List` and `Deque` provide even more ways to modify their contents. You may also remove an element using `Iterator`'s `remove` method while you are iterating through a `Collection`.

Again, the exam does not expect you to know all the methods by heart and it does not

trick you by using bogus methods. It does expect you to know the following things though:

1. The meaning of the values returned by commonly used methods. For example, when a method returns a `boolean`, a `true` value implies that the collection was indeed modified because of the method call. But `Map.put` returns the previous value associated with key, or `null` if there was no mapping for key in the map.
2. Exceptions thrown by methods declared in the `Collection` and `List` interfaces when you try to pass nulls or invalid values. For example, if you try to add/remove an element to/from a `List` at an invalid index (i.e. at a value less than 0 or greater than size-1), an `IndexOutOfBoundsException` will be thrown.
3. How the collection handles duplicates. For example, if you try to add an object to a `HashSet`, it will just return `false` if the object already exists (by using `hashCode` and `equals` methods to check).
4. Since `SequencedCollection` and `SequencedMap` are new additions to the Collections API in Java 21, you will see a question or two on their methods.

I have noted all the important methods in the tables at the beginning of this chapter. Going through the JavaDoc descriptions of those will be sufficient for the exam.

The following are a few examples of the kind of code you will see in the exam.

Adding Elements

```
Set<String> set = new HashSet<>();
set.add("a"); //[a]

ArrayList<String> sList = new ArrayList<>();
sList.add("b"); //[b]
sList.addAll(set); //elements are added at the end, so, sList now
contains [b, a]
```

Since `List` is a `SequencedCollection`, you may also do

```
sList.addFirst("a")
```

```
sList.addLast("b")
```

The `addFirst` and `addLast` methods of `List` just delegate the call to `add(0, obj)` and `add(obj)` methods respectively.

You can't "add" elements to a `Map` but you can "put" key-value pairs in it:

```
Map<Integer, String> map = new HashMap();
System.out.println(map.put(10, "Alice")); //prints null
System.out.println(map.putIfAbsent(10, "Bob")); //prints Alice
```

Observe that `put` and `putIfAbsent` return previously associated value (or `null`, if the key was absent) in the map.

Removing Elements

```
Collection<String> c = new ArrayList(List.of("a", "a", "a"));
c.remove("a");//removes only the first a
c.removeIf( v → v.equals("a")); //removes all values that satisfy the
Predicate
```

Since `List` has an overloaded remove method that takes an `int` for index, you need to be careful while removing elements from a `List<Integer>`:

```
ArrayList<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.remove(1); //passing int 1, removes element at index 1
System.out.println(list); //prints [1, 3]
list.remove(Integer.valueOf(1)); //passing an Integer object containing
1
System.out.println(list); //prints [3]
```

Recall the rules of method selection in the case of overloaded methods. When you call `remove(1)`, the argument is an `int` and since a remove method with `int` parameter is available, this method will be preferred over the other remove method with `Object` parameter because invoking the other method requires boxing `1` into an

`Integer`.

```
Collection<String> c1 = new ArrayList<>(List.of("a", "b", "c", "a"));
Collection<String> c2 = new ArrayList<>(List.of("a", "b"));
c1.removeAll(c2);
System.out.println(c1); //prints [ c ]
```

Observe that unlike the `remove(Object obj)` method, which removes only the first occurrence of an element, `removeAll` removes all occurrences of an element.

`List` has a `set(int index, E element)` method that replaces the element at the specified position in this list with the specified element. It returns the element that was replaced:

```
ArrayList<String> al = new ArrayList<>(List.of("a", "b", "c")):
System.out.println(al.set(1, "x"));  //prints old value b
System.out.println(al); //prints [a, x, c]
```

Map has two overloaded `remove` methods:

```
Map<Integer, String> map = new HashMap();
map.put(10, "Alice");
System.out.println(map.remove(10, "Bob")); //prints false
System.out.println(map.remove(10)); //prints Alice
System.out.println(map.remove(10)); //prints null
```

The first remove returns `false` because `10` is associated with `"Alice"` and not with `"Bob"`. So, even though key `10` is present in the map, it is not removed. The second remove the key 10 from the map and returns `"Alice"` because that is the existing value associated with `10` and the third remove returns `null` because there is no key `10` in the map at this point.

Using SequencedCollections

The `SequencedCollection` interface provides the following methods: `addFirst`, `addLast`, `getFirst`, `getLast`, `removeFirst`, `removeLast`, and `reversed`. The methods names clearly reflect what they do. The tricky part is to remember the classes that implement this interface.

Since `List` extends `SequencedCollection`, all lists such as `LinkedList`, `ArrayList`, and `CopyOnWriteArrayList`, implement these methods as expected.

Since `Deque` extends `SequencedCollection`, all deques such as `LinkedList` and `ArrayDeque` implement these methods as expected.

`Queue` does not extend `SequencedCollection` and so, you should watch out for code like this:

```
Queue<String> q = new ArrayDeque();
q.addFirst("first");//will not compile
```

The above code does not compile because `Queue` does not have the `addFirst` method.

Among sets, only `LinkedHashSet` and `TreeSet` implement `SequencedSet` (which extends `SequencedCollection`), however, since `TreeSet` keeps its elements sorted, it throws `UnsupportedOperationException` from its `addFirst` and `addLast` methods.

Using SequencedMaps

The methods offered by `SequencedMap` are self explanatory. You need to be aware that `LinkedHashMap` and `TreeMap` implement `SequencedMap` while `HashMap` does not. Just like `TreeSet`, `TreeMap` too throws

`UnsupportedOperationException` from its `putFirst` and `putLast` methods because it keeps its keys sorted.

18.4 Quiz ☝

Q1. What will the following code print?

```
List<String> names = new ArrayList(List.of("a", "b", "c", "d", "e"));
Spliterator<String> split1 = names.spliterator();
System.out.print(split1.hasCharacteristics(Spliterator.SIZED));
System.out.println("  "+split1.estimateSize());
Spliterator<String> split2 = split1.trySplit();
while(split1.tryAdvance((value) → System.out.print(value))){ };
System.out.println("");
split2.forEachRemaining(System.out::print);
```

Select **1** correct option.

A.

```
true 5
<exception at runtime>
```

B.

```
false <an unpredictable number>
cde
ab
```

C.

```
true 5
```

```
cde
ab
```

(Order and split of letters is unpredictable)

D.

```
true <an unpredictable number>
cde
ab
```

Correct answer is C.
Since the number of elements in the list is known, a `Spliterator` created over a List is "sized". Therefore, it prints `true` and `5`.
The `trySplit` method tries to split the existing elements in the `Spliterator` **approximately** in half. Since the exact splitting mechanism depends on the implementation, we cannot rely on which half will get how many and what elements.
The `tryAdvance` method returns `true` if the advance was successful. Therefore, the `while` loop and the call to `forEachRemaining` are basically doing the same thing.

Q2. (For OCP 21) Given:

```
SequencedCollection<String> names = //instantiate an implementation class
names.addFirst("bob ");
names.addFirst("chuck ");
names.addFirst("alice ");
names.addFirst("chuck ");
names.forEach(System.out::print);
```

Select **2** correct options.

A. If `names` is initialized with `new LinkedHashSet<>()`, the output will be: `chuck alice bob`
B. If `names` is initialized with `new TreeSet<>()`, the output will be: `alice bob`

`chuck`
C. If `names` is initialized with `new LinkedList<>()`, the code will not compile.
D. If `names` is initialized with `new ArrayList<>()`, the output will be: `chuck alice chuck bob`

Correct answer is A, D.
All four classes implement `SequencedCollection`. Therefore, there will be no compilation error. Hence, **C** is incorrect. A `TreeSet` is `SequencedCollection` but the sequence in which it keeps elements corresponds to the sorted order of the elements (instead of the order of insertion as is the case with `ArrayList` or `LinkedList`). For this reason, `addFirst` / `addLast` and other such methods don't make sense for a `TreeSet` and so, these methods throw an `UnsupportedException`, (it is an unchecked exception) at run time. Therefore, **B** is incorrect.
The `addFirst` method inserts an element in the first position pushing the existing elements in the next position. Since a `Set` stores only unique elements, adding chuck the second time in a `LinkedHashSet` will not cause any change in the set. Therefore, **A** is correct. An `ArrayList` allows duplicates. Thus, **D** is also correct.

18.5 Searching and Sorting ☝

You must have noticed that the `Collection` interface declares several methods such as `contains(Object )`, `containsAll(Collection )`, and `remove(Object )`, that essentially "search" for the given object(s) in the collection to perform their action. Searching involves checking whether two objects are equal. As explained in the Operators chapter, the relational operator `==` is good for checking whether two references are pointing to the same object or not but this does not help us in searching for an object within a collection because while searching we are more interested in logical equality than in referential equality. For example, while searching for a Person in a collection of Persons, we are not interested in checking whether the collection contains the same Person object or not but in checking whether the collection has a Person object whose first name and last name matches with the firstName and

lastName of the Person object that we are searching for. In other words, if two different Person objects have the same first name and last name, we may consider those two Person objects as logically equal.

Furthermore, some collections such as `TreeSet` keep their elements sorted. But sorting requires something more than just a check for equality. For sorting, we need to determine if one object is logically "bigger" or "smaller" than the other. For example, if you want a collection of Persons to be sorted by their dates of birth, a person with an earlier date of birth could be deemed logically bigger than the other.

Java has three ways to perform logical comparison and you need to know all three of them before learning about searching and sorting.

18.5.1 The equals method ☝

The most basic way of checking logical equality (aka object equality) is to use the "equals" method. This method is available in all classes because it is implemented in the `Object` class. Since it is available for all objects, even the `Collection` interface specifies that the methods that require searching for an object check logical equality using this method. However, the `Object`'s version of this method isn't too helpful because it just compares the references using `==`.

So, if you expect to compare two objects of a class based on their contents, then you need to override the `equals(Object obj)` method in that class and write the logic that does the comparison in it. For example, if you, as a developer of a `Person` class believe that two `Person` objects are logically equal when their first and last names are the same, then you have to code this logic in the `equals` method of the `Person` class:

```
class Person{
  String firstName;
  String lastName;
  @Override
  public boolean equals(Object otherPerson){
    if(otherPerson instanceof Person op){
```

```
      return firstName.equals(op.firstName)&& lastName.equals(op.lastName);
    } else return false;
  }
}
```

Observe the following points about the `equals` method written above:

1. It is annotated with `@Override`. Although not required, it makes our intention, that we are trying to override the `equals` method, clear. In this case, `Object` is the direct super class of `Person`, which means, we are overriding the `equals` method of the `Object` class.
2. It is **public**. The `equals` method in `Object` is public and since an overriding method cannot reduce the accessibility of the overridden method, we must make it `public`.
3. The parameter type of the method is `Object` (and not `Person`) because the parameter type of the overridden method is `Object`. Recall that Java does not support covariant parameter types. So, if you change the parameter type to `Person`, it will be a overload and not an override and the `@Override` annotation will cause the compiler to generate an error.

Many classes in the Java standard library override `equals` but some don't. For example, `String` overrides `equals` to compare the contents but `StringBuilder` doesn't. Although the exam does not require you to know this for all classes, you should to refer to the JavaDoc of the class before relying on its `equals` method for checking logical equality. In practice, entity classes, data transfer objects, or value objects, need to implement the equals method to check for logical equality. Since these classes have many fields, it becomes quite tedious to code this method. Java's record classes are helpful in this respect because the compiler automatically generates such an `equals` method for a record class.

The hashCode method ☝

Although the exam does not have questions on this method, a discussion on `equals` is incomplete without talking about hash code.

If you are not from Computer Science background, a **hash code** is just a numeric value generated by applying some function or some technique on a piece of data. For example, I could use the length of a string as my function for generating the hash code for strings. So, the hash codes for `Amy` and `Bob` would be `3` but for `John`, it would be `4`. As you can see, multiple strings may have the same hash code, and that is fine. Mathematically, a hashing function is a many-to-one function that maps elements of a larger set to elements of a smaller set.

In Java, a hash code of an object is the `int` value returned by the `hashCode()` method of that object. This value is usually generated by using the data stored in the fields of an object and is kind of a short representation of that object. It is helpful when we have to search for an object in a set of a million objects. Instead of comparing all the fields of all the objects, you we can narrow the search first by comparing the hash codes of the objects. If an object with the same hash code is found, we can then compare the fields to make sure that the two objects are indeed logically the same. In the above example, if we want to search for `Bob` among a list of million names, we can first narrow the search to just those names whose hash code is `3` and then check for the actual name in the resulting, hopefully, a lot smaller, set of names. The `HashSet` and `HashMap` classes use this mechanism to increase their search performance, which is why understanding the relationship between `equals` and `hashCode` is important.

Just like the `equals` method, the `Object` class implements the `hashCode()` method as well. However, the `Object` class's `hashCode()` generates a completely unique value for each object. This is consistent with `Object` class's version of the `equals` method, which deems every object to be unique (remember that `Object`'s `equals` performs referential equality using `==`) but it is entirely useless when searching for a logically equal object in a `HashSet` or a `HashMap`. If we try to use `Object`'s `hashCode` while checking for logical equality, we will never find a logically equal object in a `HashSet` or a `HashMap` unless we are searching for the exact same object in memory. We may find it in non-hash based collections such as `ArrayList` because they do not use hash codes to narrow the search space but then they are not as fast as a hash based collection either.

So, the rule that we must follow is that if two objects are deemed equal by their `equals` method, then their `hashCode()` method must also return the same `int` value. This means that if we override the `equals` method in a class to implement logical equality, we should also override the `hashCode` method. It is not a surprise therefore, that the compiler generates a `hashCode` method as well along with the `equals` method for a record class.

18.5.2 The Comparable interface ☝

A class that knows how to order its objects should implement the `java.util.Comparable` interface. Implementing `Comparable` signifies that objects of this class are comparable to each other. It also establishes the "natural order" of the objects of this class. Natural order means that we can order the elements by just using the properties of those objects without any external information or logic. For example: numbers have a natural order because we know which number is larger or smaller by looking at their magnitudes. But if the `Person` class does not implement `Comparable`, `Person` objects will not have any natural order and data structures such as `TreeSet` and `TreeMap` that keep their objects sorted will not be able to work with `Person` objects. Specifically, adding a `Person` object to a `TreeSet` will throw a `ClassCastException` at run time because `TreeSet` expects `Person` to be `Comparable`.

The `Comparable` interface has only one method: `int compareTo(T o)`. A positive return value indicates that this object is logically bigger than the object passed as the argument and a negative value indicates the opposite. A zero return value indicates that the two objects are logically equal.

Many classes in the Java standard library implement this interface. In fact, most of the data oriented classes such as `String`, `StringBuilder`, and all primitive wrapper classes implement `Comparable`. This is great because we can use their implementations of `compareTo` while implementing `compareTo` in our class.

Let's see how we can make our `Person` class `Comparable`:

```
class Person implements Comparable<Person>{
    String name;
    int age;

    public int compareTo(Person otherP) {
        int a = this.age - otherP.age;
        if(a != 0) return a; //this establishes the order based on age
        else return name.compareTo(otherP.name); //if the age is same
then we order alphabetically
    }

    public boolean equals(Object o){
        if( o instanceof Person p){
            return age == p.age && name.equals(p.name);
        }else return false;
    }
}
```

As per the implementation above, we consider two `Person` objects to be equal only when their names and their ages match. The `compareTo` method also uses the same logic to determine if two `Person` objects are equal. Observe that in our implementation of `compareTo`, we are relying on `String`'s `compareTo` and that is okay because `String` does implement `Comparable`.

Although not technically required, it is important to keep the implementation of `compareTo` and `equals` consistent with each other. Meaning, `a.compareTo(b)` should return `0` if and only if `a.equals(b)` returns `true`. If you don't follow this restriction, then you will essentially have two different definitions of logical equality for that class and that may introduce hard to trace bugs as illustrated by the following code:

```
//change compareTo of Person to compare just the age
```

```
public int compareTo(Person otherP) {
   return this.age - otherP.age;
}

public static void main(String[] args){
   Person p1 =  new Person(); p1.age = 20; p1.name = "John";
   Person p2 =  new Person(); p2.age = 20; p2.name = "Bob";
   System.out.println(p1.equals(p2)); // prints false

   Set<Person> hSet = new HashSet<>();
   hSet.add(p1); hSet.add(p2);
   System.out.println(hSet.size()); //prints 2

   Set<Person> tSet = new TreeSet();
   tSet.add(p1); tSet.add(p2);
   System.out.println(tSet.size()); //prints 1

   Person p3 =  new Person(); p3.age = 20; p3.name = "Charlie";
   System.out.println(hS.contains(p3)); //prints false
   System.out.println(tS.contains(p3)); //prints true
}
```

In the above code, we are adding two `Person` objects that are unequal as per the `equals` method to two sets. Since the objects are unequal, we expect both the sets to contain two objects. However, only the `HashSet` works as expected. The `TreeSet` ignores the second addition because as far as `TreeSet` is concerned, the two `Person` objects are equal because `p1.compareTo(p2)` returns `0`. Furthermore, since `TreeSet` uses the `compareTo` method to check for equality, `tS.contains(p3)` returns `true` but `hS.contains(p3)` returns `false`.

18.5.3 The Comparator interface ☝️

If it is not possible for a class to implement `Comparable` (this could happen if you don't control its source code) and if you still want to keep the objects of this class in sorted data structures, then the `Comparator` interface is what you need. A `Comparator` allows us to take the logic of ordering out from the class whose objects we want to order and put it into an independent class. It has only one abstract method `int compare(T o1, T o2)` which should contain the logic for comparison. Here is how:

```
class PersonComparator implements Comparator<Person>{
  public int compare(Person p1, Person p2) {
    int a = p1.age - p2.age;
    if(a != 0) return a; //this establishes the order based on age
    else return p1.name.compareTo(p2.name); //if the age is same then we
order alphabetically
  }
}
```

We can now use this `Comparator` to enforce an order expected by a `TreeSet` or a `TreeMap` by passing an instance of the comparator to their constructors, for example: `Set<Person> s = new TreeSet<Person>(new PersonComparator());`. The `TreeSet` will not attempt to cast added objects to `Comparable` anymore. It will use the `Comparator` to determine the position of the objects.

Note that the guideline about keeping `equals` and `Comparable`'s `compareTo` applies to `equals` and `Comparator`'s `compare` as well but it is not as critical to follow because you will shortly see that a `Comparator` can also be used to sort the objects without the involvement of sorted data structures.

In addition to the `compare` method, `Comparator` has a ton of **default** as well as **static** methods that help us build new comparators by either transforming the ones that you have already created or by reusing readymade comparators provided in the Java library. For example, if you want to order `Person` objects in the reverse order, then you can call the default method `reversed()` on an existing

`PersonComparator` instance to get a new `Comparator` as follows:

```
Comparator<Person> reversed = new PersonComparator().reversed();
```

If you want to create a `Comparator` that orders `Person` objects based only on their ages, you could use the static method `comparingInt(ToIntFunction f)`:

```
Comparator<Person> ageComparator = Comparator.comparingInt(p→p.age);
```

There are methods that allow you to chain one comparator after another as well. For example, if you want to order `Person` objects first by their names and then, if the names are the same, on their ages, you can build a new `Comparator` by chaining two comparators one after another using the default method `thenComparing` as follows:

```
Comparator<Person> nameComparator = (pA, pB)→pA.name.compareTo(pB.name);
Comparator<Person> nameAndAgeComparator = nameComparator.thenComparing(
ageComparator );
```

Observe that I have implemented `nameComparator` using a lambda expression. This is possible because `Comparator` is a functional interface.

For the purpose of the exam, the above three methods are sufficient but do take a look at `Comparator`'s JavaDoc to see other similar methods as they will come in handy while coding professionally.
Observe that `Comparator` is a genuine functional interface not just because it has exactly one abstract method but because it makes the functionality usable independently from the class that stores the data. On the other hand, `Comparable` too has exactly one abstract method and is also technically a functional interface but logically, `Comparable` describes the inherent natural order of

the objects of a class by keeping the functionality collocated within the class whose objects it compares and that is why `Comparator` is annotated with `@FunctionalInterface` while `Comparable` is not.

18.5.4 Quiz ☝

Q. Which of the following `Comparator` implementations correctly order two strings in the increasing order of their lengths?

Select **1** correct option.

A. `Comparator<String> c = (a, b)→{ return a.length()-b.length(); };`

B. `Comparator<String> c = (a, b)→b.length()-a.length();`

C. `Comparator<String, String> c = (a, b)→b.length()-a.length();`

D. `Comparator<String, String> c = (a, b)→a.length()-b.length();`

Correct answer is A.

Conceptually, while comparing two objects `a` and `b`, a comparator considers `a` (that is, the first argument) to be the "smaller" of the two if "a minus b" is negative. In this case, we want to compare two strings `a` and `b` based on their lengths such that `a` is smaller than `b` if `a`'s length is smaller than `b`'s. Therefore, we must do `a.length() - b.length()` and not `b.length()-a.length()`. Hence, **A** is correct and **B** is not. **B** is syntactically correct but it will order the strings in decreasing order of their lengths.

`Comparator` has only one type parameter (because it is meant to compare two objects of the **same type**). Thus, options **C** and **D** will not compile.

18.5.5 Sorting and Searching Lists ☝

Collections that don't keep elements sorted by default such as `ArrayList`, `LinkedList`, `ArrayDeque`, `HashSet`, and `HashMap`, can be searched for an element by using the `equals` method to compare the required object with objects in that collection. Exactly how many comparisons will be done for a given search depends on how the data structure of the collection manages the objects internally. For example, in case of an `ArrayList`, the given object will be tested against every object in the list one by one until a match is found. This is called **Linear Search**. In technical terms, we say that its time complexity is `O(n)`. It is considered slow.

A `HashSet` does this more efficiently because, as explained earlier, it reduces the search space first by using the hash code and then using the `equals` method to find an exact match. It's time complexity is `O(1)` because if a good hashing function is used, every object will have a different hash code and only one comparison will be needed to find a match. It is considered very fast. But hash based data structures use more space because they have to store hash codes as well along with the object and there is an added restriction that you can't store duplicate values in a `Set`.

What if you want to store duplicate elements and also want a fast search? Well, you can use a `List` that is already sorted. Searching through a sorted list can be done using the **Binary Search** algorithm, which is substantially faster than Linear search. It's time complexity is `O(log n)`.

The `java.util.Collections` class is a utility class that provides several static methods to sort and search for objects in a `List`. Let's look at them one by one.

The `Collections.sort` methods ☝

There are two variants of this method, both of which sort the list in the **ascending** order: `static <T extends Comparable<? super T>> void sort(List<T> list)` and `static <T> void sort(List<T> list, Comparator<? super T> c)`
The following are the important features of these methods that you need to know for the exam:

1. They return void.
2. They sort the given list itself. They do not return a new sorted list.
3. The sort is guaranteed to be **stable**, meaning, equal elements will not be reordered as a result of the sort.
4. The given list must be modifiable.
5. The given list may contain duplicates but must **not** contain **nulls**. A `NullPointerException` is thrown otherwise.
6. The single argument version sorts the elements by comparing them using `Comparable`'s `compareTo` method. This means, the class of the objects present in the list must implement `Comparable`.
7. The two arguments version sorts the elements by comparing them using `Comparator`'s compare method. This means, the class of the objects present in the list need not implement `Comparable`.

The following code illustrates the usage of both the methods:

```
List<String> names = new ArrayList<>(List.of("bbb" , "10", "9",  "B",
"aa", "a"));
Collections.sort(names);
System.out.println(names); //prints [10, 9, B, a, aa, bbb]
Collections.sort(names, (s1, s2) -> s1.length()-s2.length());
System.out.println(names); //prints [9, B, a, 10, aa, bbb]
```

Observe the following points in the above code:

1. I am creating a new `ArrayList` because `List.of` returns an unmodiable `List` and I need to pass a modifiable list to the sort method.

2. Since `String` implements `Comparable`, I am able to sort the list without passing in a `Comparator`.
3. Although the number `10` is larger than the number `9`, it appears before `9` in the output because the array being sorted is not of numbers but of strings. Since String's `compareTo` method compares strings lexicographically, `"10"` is considered smaller than `"9"`. Furthermore, as characters, arabic numerals appear before English upper case letters and the English upper case letters appear before English lower case letters, which means, `1<9<B<a`.
4. In the second sort, I am using a lambda to create a `Comparator` that compares two strings on their lengths. Further observe that the order of elements `9`, `B`, and `a` does not change during the second sort because the sort is guaranteed to be **stable**.

The `java.util.Collections` class has been around since Java 1.2 and it has had the `sort` methods since then. However, after default methods in interfaces were introduced in Java 1.8, the logic of sorting a list was moved into `List`'s default `sort(Comparator )` method and `Collection`'s sort methods were made to just delegate the call to `List`'s `sort` method, which means, you can sort a `List` directly without using the `Collections` class. For example,
`names.sort(null);` //passing null Comparator, Comparable's compareTo will be used
`names.sort((s1, s2) → s1.length()-s2.length());` //passing non null Comparator

The `Collections.binarySearch` methods ☝

There are two variants of this method as well:
`static <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)` and

`static <T> int binarySearch(List<? extends T> list, T key, Comparator<? super T> c)`

The following are the important features of these methods that you need to know for the exam:

1. They return an `int` telling you the index at which the search key was found. If the return value is zero or positive, it means that the key was found at that index. If the return value is negative, it tells you the insertion point, meaning that the element was not found in the list but if you were to add it to the sorted list, it would be placed at position given by the formula `-(return value + 1)`. For example, if it returns `-1`, that means the key would be inserted at `-(-1+1)` i.e. at the zeroth index in the list.
2. They list must already be sorted in **ascending** order. Otherwise, the return value is undefined.
3. If the list has duplicates, there is no guarantee which one's index out of the matching ones will be returned.
4. The given list must **not** contain **nulls**. A `NullPointerException` is thrown otherwise.
5. The two argument version searches for the key using `Comparable`'s `compareTo` method. This means, the class of the objects present in the list must implement `Comparable`.
6. The three arguments version searches for the key using `Comparator`'s compare method. This means, the class of the objects present in the list need not implement `Comparable`.

The following code illustrates the usage of the two methods:

```
List<String> names = new ArrayList<>(List.of("bbb" , "a", "b", "aa", "a"));
Collections.sort(names); //sorted as [a, a, aa, b, bbb]
System.out.println(Collections.binarySearch(names, "aaa")); //prints -4
```

```
Comparator<String> c = (s1, s2) → s1.length()-s2.length();
Collections.sort(names, c); //sorted as [a, a, b, aa, bbb]
System.out.println(Collections.binarySearch(names, "x", c)); //prints 2
```

In the above code, the first search returns `-4` because `aaa` is not found in the list. But if I add `aaa` to the given list and sort the list, I would find `aaa` at index `-(-4+1)`, i.e., `3` (after `aa`) in the sorted list. Observe that in the second search, I am first sorting the list using a custom `Comparator` that compares strings based on their lengths, and then I am using the same `Comparator` to search. It prints `2` because `x` (whose length is 1) is found in the list at index `2`. Although elements at index `0` and `1` are also a match, the method does not guarantee which element's index will be returned. It could theoretically return `0` or `1` too.

Other important methods in the Collections class ☝

Besides the `sort` and `binarySearch` methods, the following is a list of methods in the `Collections` class that you should be aware of for the exam:

1. `public static void reverse(List<?> list)`: Reverses the order of the elements in the specified list. The given list must be mutable for this method to work because it modifies the given list.
   If you just want to have a reversed view of an existing list without mutating it, use `List`'s default method `reversed()`.

So, far, I have only heard of the `reversed()` method appearing in the exam.

18.5.6 Quiz ☝

Q1. What will the following code print?

```
List<String> names = new ArrayList(List.of("a3", "a1", "a2"));
```

```
System.out.print(Collections.binarySearch(names, "a1")+" ");

Collections.sort(names);
System.out.print(Collections.binarySearch(names, "a1")+" ");

Collections.reverse(names);
System.out.print(Collections.binarySearch(names, "a1")+" ");
```

Select **1** correct option.

A. `-1 0 -1`
B. `1 1 -2`
C. <Output cannot be predicted> `1` <Output cannot be predicted>
D. `1 0 -1`

Correct answer is D.
Although, if you run the given code, it will print 1 0 -1. However, as per the JavaDoc, the result of `binarySearch` is deterministic only if the given list is **sorted in ascending** order. Since the given list is initially unsorted, the output for the first search is undefined. Once it is sorted, the output will be `0` because `a1` will be found at the 0th position. Finally, when the list is reversed, the elements of the list will not be in ascending order and so, the output will again be undefined.

18.5.7 Sorting and Searching arrays ☝

Just as the `java.utils.Collections` class contains several utility methods for dealing with collections, so, does the `java.util.Arrays` class contains methods for dealing with arrays, including sorting and searching. I will list the important methods of this class with brief descriptions below.

1. Overloaded static sort methods. For example, `static void sort(byte[]`

1. `a)` and `static void sort(byte[] a, int fromIndex, int toIndex)`. Besides the sort methods for `byte[]`, it has similar sort methods for `int[]`, `char[]`, `short[]`, `float[]`, `double[]` and `Object[]`. They sort the specified array into ascending natural order. The `sort(Object[] )` method expects that the class of the objects implements the `Comparable` interface.

   It also has `static <T> void sort(T[] a, Comparator<? super T> c)` and `static <T> void sort(T[] a, int fromIndex, int toIndex, Comparator<? super T> c)` that sort the specified array of objects according to the order induced by the specified comparator.
   Just like `Collections.sort`, `Arrays.sort` methods throw a `NullPointerException` if any element of the input array is `null`.

2. Overloaded static `binarySearch` methods for arrays of all primitive data types as well as for array of Objects. Their return value is either the index of the matched element or `-(insertion point) - 1`. Just like the `Collections.binarySearch` method.

3. `static <T> List<T> asList(T ... a)` - This method returns a fixed sized list view of the array. Since it is backed by the same array that you pass in, you may change the elements of the list but you can't change its size. So, the `set(index, object)` will work but `add` and `remove` methods will throw an `UnsupportedOperationException`. For the same reason, any changes that you make to the underlying array will be visible in the list created from that array using this method.

4. Overloaded static `stream` methods that provide a `Stream` view of the array.

5. Overloaded static `spliterator` methods that return a `Spliterator` view of the array.

6. Static `compare` and `mismatch` methods, which I have already discussed in detail in "Creating and Using Arrays" chapter.

7. Overloaded static `toString` methods that return a convenient string representation of an array of primitive as well as reference types. For example, `Arrays.toString(new String[]{"a", null, "c"})` returns `[a, null, c]`.

18.5.8 Quiz ☝

Q 1. What will the following code print?

```
String[] sa = {"4", "-4", "14", "a", "Aaaa"};
Arrays.sort(sa);
System.out.println(Arrays.toString(sa));
```

Select **1** correct option.

A. `[-4, 4, 14, Aaaa, a]`

B. `[14, 4, Aaaa, a, -4]`

C. `[-4, 14, 4, a, Aaaa]`

D. `[-4, 14, 4, Aaaa, a]`

The correct answer is D.

`Arrays.sort(Object[] )` method sorts the elements of the array using the `compareTo` method of the class of the objects. Here, it uses the `compareTo` method of the `String` class, which performs lexicographical comparison. As per the lexicographical order of the minus sign, digits, upper and lower case English letters, option **D** has the correct order of the given strings.

Q 2. Given:

`String[] students = { "Bob", "Dave", "Chris", "Andy" };`

Determine the output produced by the following code fragments:

A.

```
List<String> list = List.of(students);
list.sort((a, b) → a.compareTo(a));
System.out.println(Collections.binarySearch(list, "BOB"));
```

B.

```
List<String> list = Arrays.asList(students);
list.sort(null);
System.out.println(Collections.binarySearch(list, "Bob"));
```

C.

```
List<String> list = Arrays.asList(students);
list.replaceAll(s→s.toUpperCase());
list.sort(null);
System.out.println(Collections.binarySearch(list, "bob"));
```

D.

```
List<String> list = Arrays.asList(students);
Arrays.sort(students);
list.removeIf(s→s.equals("Bob"));
System.out.println(Collections.binarySearch(list, "bob"));*
```

Answer:
A. `List.of` and `List.copyOf` methods create an immutable list and all the methods of List that attempt to modify the list in any way (including the `sort` method) throw a `java.lang.UnsupportedOperationException`.

B. `Arrays.asList` produces a list that is backed by the array and so, all the modifications that are permitted on arrays are permitted on the list. This means,

operations such as `sort` can be performed on the list. However, operations such as `add` and `remove` are not allowed.

`List`'s `sort` method takes a `Comparator`. If the argument is `null`, objects in the list must be `Comparable` and the objects are compared using their `compareTo` method. In this case, since the list is of `String`s, all the elements of the list will be sorted lexicographically. String `bob` will be at index 1 in the sorted list.

C. In this code, all the elements of the list are replaced by strings containing only uppercase. In lexicographical comparison, uppercase English letters appear **before** lowercase English letters and so, the string `bob`, if inserted in this list, will be inserted at index 4 (after `DAVE`). Therefore, the search for `bob` will return -(4) - 1, i.e., `-5`.

FYI, the order of ASCII characters in a lexicographical comparison is : the minus/dash/hyphen sign < digits 0 to 9 < uppercase English letters < lowercase English letters. So, for example, a search for `-bob` in this list will return -(0) - 1, i.e., `-1`.

D. Since it is not possible to add or remove elements to/from an array, a list created using `Arrays.asList` does not allow operations that attempt to add or remove elements. Therefore, the `list.replaceIf` method will throw an `UnsupportedOperationException`.